DEVOPS MINI PROJECT

Subject: DevOps

Automated Deployment of a Student Record and Academic Management Platform on AWS

Student Name	Nandini Dhonde
PRN	2125UDSF1072
Subject	DevOps

Infrastructure Automation and Configuration Management

1. Executive Summary

This mini project describes a repeatable way to host a Student Record and Academic Management Platform in AWS. It combines Terraform for defining cloud resources with Ansible for preparing the server and deploying its services. Docker runs the application in a packaged environment, and Nginx routes browser traffic to the application. The report outlines the design, automation sequence, checks, and security practices suitable for a student demonstration.

Project Aim

To reduce repeated manual server setup by describing cloud infrastructure and server configuration in files that can be reviewed, reused, and tested.

2. Requirements Analysis

Before creating cloud resources, the application's runtime, network path, storage needs, and access rules should be identified. The following mapping gives a small, single-server design for coursework; it is not intended to represent a highly available production environment.

Application / Need	Selected Component	Purpose
Student API	Ubuntu EC2 instance	Runs the application host and supporting services.
Incoming web requests	Nginx on port 80	Accepts HTTP traffic and forwards requests to the application.
Application runtime	Docker container on port 5000	Keeps the app and its runtime dependencies together.
Academic sample records	Demo data store / persistent volume	Stores example student and marks data between restarts when configured.
Network boundary	VPC, subnet, route table, Internet Gateway	Defines where the instance sits and how internet routing works.
Access control	EC2 Security Group	Allows only the network ports required for administration and web access.
Project files	Git repository	Tracks Terraform files, Ansible playbooks, and application source.

2.1 Software and Tools

- AWS account with permission to create the selected resources; review current charges before launching anything.
- Terraform CLI and the AWS provider.
- Ansible and SSH access from a control machine.
- Ubuntu Server image, Docker Engine, Nginx, and a working application image or source tree.
- Git and a GitHub repository for versioned project files.

3. Proposed AWS Architecture

A browser sends a request to the EC2 public address. Nginx receives the request and forwards it to the Student API running in Docker. Terraform defines the network and compute resources, while Ansible configures the operating system and application services after the host is reachable.

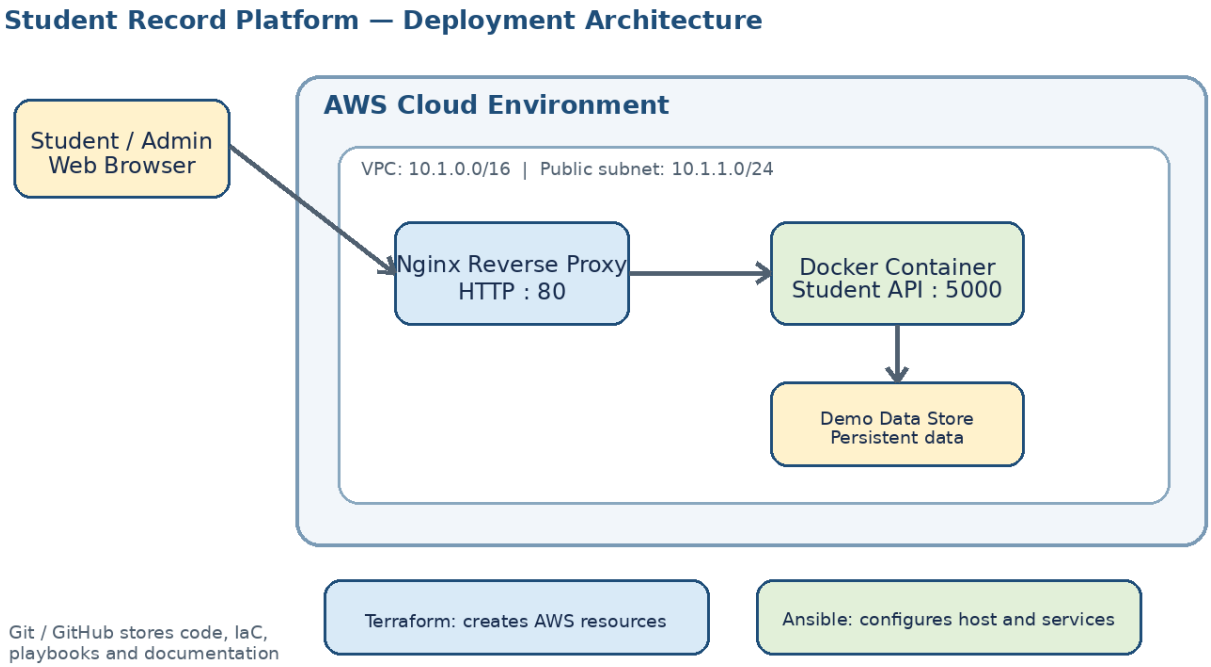


Figure 1. Proposed deployment layout for the Student Record Platform.

3.1 Infrastructure Components

Component	Configuration / Role
VPC	10.1.0.0/16 network range for the project.
Public subnet	10.1.1.0/24 subnet for the demonstration instance.
Internet Gateway and route table	Provide a route to the internet when security rules and routing are correctly configured.
EC2	Ubuntu server sized for a small demonstration; select availability and instance type based on current AWS options.
Security Group	Permit HTTP on port 80; restrict SSH on port 22 to the administrator's IP. Keep port 5000 private where possible.
Optional backup storage	Private S3 bucket with encryption and suitable retention/versioning controls.

4. Infrastructure Definition with Terraform

Terraform lets the project owner describe the intended cloud resources in configuration files. It uses the AWS provider to communicate with AWS APIs, tracks the resources it manages in state, and presents a plan before applying changes.

Command	Purpose
terraform init	Initializes the working directory and retrieves required provider components.
terraform fmt	Applies consistent formatting to Terraform configuration.
terraform validate	Checks configuration structure and expression validity.
terraform plan	Shows a preview of proposed resource changes.
terraform apply	Creates or updates the resources after confirmation.
terraform destroy	Removes resources managed by the configuration when the environment is no longer needed.

4.1 Resources to Describe

- VPC and public subnet.
- Internet Gateway, route table, and subnet association.
- Security Group with narrowly scoped inbound rules.

- EC2 instance and required instance profile or permissions, if needed.
- Outputs such as the instance public address for the configuration stage.
- Optional private backup bucket, configured with suitable protections.

4.2 State and Team Safety

Terraform state links configuration to real cloud objects and can reveal infrastructure details. Keep it out of public source control. For shared environments, use a secured remote backend and a locking approach supported by the selected backend and Terraform version. Review every plan before applying it.

5. Server Setup with Ansible

Ansible connects to the target machine through SSH and applies tasks from YAML playbooks. The same playbook can be run again to check that the server still matches the intended configuration. Tasks should be written to avoid needless changes on later runs.

Suggested Role	Tasks
common	Install basic tools, update package metadata, and apply baseline host settings.
docker	Install Docker Engine and enable the service.
nginx	Install Nginx and configure a reverse proxy to the application.
student_app	Configure the app, start the container, mount required persistent data, and check the health route.

5.1 Inventory and Connection

The inventory identifies the EC2 host and its SSH login user. The private key should remain on the administrator's machine or a protected secret store; it must not be committed to GitHub. Access should be tested before running the full playbook.

6. Terraform–Ansible Deployment Sequence

Stage	Action	Expected Check
1. Prepare	Review the application and commit project files to Git.	Repository contains code and configuration, not secrets.
2. Provision	Run Terraform init, fmt, validate, and plan; then apply.	AWS resources appear with the intended network settings.
3. Discover host	Read the EC2 address from Terraform output and add it to Ansible inventory.	SSH connection works from the authorized control machine.
4. Configure	Run the Ansible playbook for packages, Docker, Nginx, and the application.	Services are installed and started.
5. Verify	Call the health and sample API endpoints.	Responses match the application's documented format.
6. Clean up	Destroy test resources when finished, after preserving any required work.	Unused resources are removed and further charges are avoided.

6.1 Handoff Between Tools

Terraform can output the newly created instance address. That value is then used to identify the Ansible target. This avoids typing an address into multiple places and makes a fresh deployment easier to repeat.

7. Environment Consistency

Development, testing, and production should be described through shared configuration wherever possible. Environment-specific values—such as instance sizing, log level, and application settings—can be separated into variables or Ansible group variables. The same application and configuration checks should be used across environments.

- Keep reusable infrastructure definitions separate from environment-specific values.
- Use version control and review changes before deployment.
- Use a test environment to check configuration changes before production.
- Do not assume a single public EC2 instance provides production-grade availability.

8. Deployment Validation and Security

A deployment is not complete merely because Terraform reports that resources were created. The host, services, container, and API should be checked independently.

Validation Layer	What to Check	Example Method
Network / OS	Instance responds and SSH is restricted.	Connect with SSH from an allowed IP and inspect system status.
Services	Docker and Nginx are running.	Use system service status commands or Ansible service checks.
Container	Application container is running and port mapping is correct.	Inspect running containers and application logs.
API health	Health route returns the expected response.	Request GET /api/health from a permitted client.
Student records	Sample student endpoint responds correctly.	Request GET /api/students and inspect the response.
Marks entry	Valid test data is accepted and invalid data is handled.	Test POST /api/marks using the app's documented payload.

8.1 Security Checklist

- Limit SSH to a trusted administrator IP rather than allowing all internet addresses.
- Keep application port 5000 inaccessible from the public internet unless specifically required.
- Use HTTPS and a valid certificate before handling real users or sensitive records.
- Apply least-privilege AWS permissions and protect all credentials and private keys.
- Do not commit passwords, tokens, private keys, or Terraform state to a public repository.
- Protect backup storage with private access, encryption, and an appropriate retention policy.
- Review AWS pricing and remove resources that are no longer needed.

9. Student Implementation: Academic Records Platform

The student version uses the same automation ideas for a different application domain: student details, course enrollment, and marks. The exact API paths and data format depend on the application code selected for implementation.

Project Item	Student Platform Design
Application domain	Student profiles, course enrollment, and academic marks.
Network range	VPC 10.1.0.0/16 with subnet 10.1.1.0/24.
Access ports	SSH 22 for restricted administration; HTTP 80 for Nginx; application port 5000 kept internal where possible.
Container	Student API packaged with Docker.
Example checks	GET /api/health, GET /api/students, POST /api/marks.
Submission evidence	Terraform files, Ansible roles/playbook, architecture diagram, test output, and repository URL.

9.1 Evaluation and Deliverables

Deliverable	Evidence to Include
Terraform configuration	Resource definitions, variables, outputs, and successful validation/plan output.
Ansible automation	Inventory template, playbook, reusable roles, and repeat-run observations.
Deployment proof	Screenshots or logs showing the server, services, container, and API checks.
Technical report	Architecture, method, testing notes, limitations, and GitHub repository link.

10. Expected Outcome

When the workflow is implemented correctly, Terraform should create the selected AWS resources, Ansible should configure the EC2 host, and Docker should run the student application behind Nginx. The test checklist provides evidence for whether the service is reachable and responding. Record real command output and screenshots after performing the steps; do not describe unexecuted tests as successful.

11. Limitations and Further Work

- One instance does not provide automatic failover or high availability.
- A sample data store is not appropriate for important academic information without proper security and backup design.
- Monitoring, alerting, automated HTTPS, and CI/CD can be added as extensions.
- Future improvements may include a managed database, private application subnet, load balancer, auto scaling, log aggregation, and infrastructure security scanning.

12. Conclusion

This mini project shows how Terraform and Ansible can be used together to automate cloud provisioning and server configuration. Docker packages the application runtime, while Nginx routes incoming requests. A structured validation process helps identify problems at the network, service, container, and API levels. The design provides a practical starting point for learning cloud automation through a student records use case.

13. References

- HashiCorp Terraform documentation: https://developer.hashicorp.com/terraform/docs
- AWS Provider for Terraform: https://registry.terraform.io/providers/hashicorp/aws/latest/docs
- Ansible documentation: https://docs.ansible.com/
- AWS EC2 User Guide: https://docs.aws.amazon.com/ec2/
- AWS VPC User Guide: https://docs.aws.amazon.com/vpc/
- Docker documentation: https://docs.docker.com/
- Nginx documentation: https://nginx.org/en/docs/